



**MANIPAL
UNIVERSITY JAIPUR**

University under Section 2(f) of the UGC Act

Centre for Distance & Online Education

Manipal University Jaipur

Internal Assignment Submission – April 2025

Programme: Bachelor of Computer Applications (BCA)

Semester: IInd

Course Code & Name: DCA –1207 DATA STRUCTURES

Assignment Sets: Set-1 & Set-2

Session: April 2025

Credits: 4

Submitted By:

Name: Hirdyansh Varshney

Roll Number: 2414500172

Email ID: 2414500172@muonline.edu.in

Declaration:

I hereby declare that the content of this assignment is original and has been prepared by me.

It has not been copied from any source, and plagiarism has been strictly avoided.

Signature: Hirdyansh Varshney

Set–1

Question 1:

Answer : When designing algorithms, it's important to evaluate their efficiency and practicality. This is where the concept of algorithm complexity comes in. It helps analyze how much time and space (memory) an algorithm will require to solve a problem. By understanding complexity, we can choose the best algorithm among multiple solutions.

- **What is Algorithm Complexity?**

Algorithm complexity refers to the amount of resources an algorithm uses as the input size increases. These resources include:

- **Time complexity** – How much time an algorithm takes to complete.
- **Space complexity** – How much memory an algorithm needs during execution.

1. Time Complexity:

Time complexity measures the number of operations performed by an algorithm based on the size of the input (n). It tells us how the running time increases as the input grows.

- **Common Time Complexities:**

Time Complexity	Description	Example
$O(1)$	Constant time	Accessing an array element
$O(n)$	Linear time	Traversing an array
$O(n^2)$	Quadratic time	Nested loops (e.g., Bubble Sort)
$O(\log n)$	Logarithmic time	Binary Search

Example:

If an algorithm loops n times to search an element, it has $O(n)$ time complexity (linear).

If it divides the data in half each time (like in binary search), it is $O(\log n)$ (logarithmic).

2. Space Complexity:

Space complexity refers to the total memory used by an algorithm, including:

- Input data
- Temporary variables
- Recursive stack space

Example:

If an algorithm creates a new array of size n , its space complexity is $O(n)$.

If it uses only a few variables, the space complexity may be $O(1)$ (constant).

- **Why Complexity Matters?**

- **Performance comparison:** Helps in choosing the best algorithm.
- **Predictability:** Tells us how the algorithm will behave with large data.
- **Optimization:** Saves time and memory resources in real-world applications.

Conclusion:

Understanding algorithm complexity is essential for efficient programming. Time complexity helps estimate an algorithm's speed, while space complexity helps evaluate memory usage. A good algorithm should ideally have low time and space complexity, making it faster and more efficient for large-scale problems.

Question 2 :

Answer : Arrays are a fundamental data structure used to store elements in a linear, indexed format. A common operation is to search for a specific number in an array and replace it with another value. The following algorithm performs this operation.

Algorithm: Find and Replace in Array

Problem Statement:

Given an array of n integers, search for a specific number (target), and replace it with a new number (replacement), wherever it is found.

Algorithm Steps:

Algorithm FindAndReplace(Arr, n, target, replacement)

Step 1: Start

Step 2: Repeat for $j = 0$ to $n-1$

If $A[j] == \text{target}$ then

$A[j] \leftarrow \text{replacement}$

Step 3: End Loop

Step 4: Print the updated array

Step 5: Stop

Explanation of Steps:

- Step 1: Begin the algorithm.
- Step 2: Traverse the array using a loop from index 0 to $n-1$.
- Step 3: Check if the current element is equal to the target value.
- Step 4: If it matches, replace it with the given replacement value.
- Step 5: Continue checking all elements.

- Step 6: Print the updated array to verify the change.

Example Execution:

Input:

A = [12 , 13 , 14 , 15 , 16]

Target = 14

Replacement = 20

Output:

Updated Array = [12, 13, 20 , 15, 16]

- **Time & Space Complexity:**
 - **Time Complexity:** $O(n)$ – We scan each element once.
 - **Space Complexity:** $O(1)$ – No extra space used, replacement done in-place.
 - **Use Case:**

This algorithm is commonly used in scenarios like:

- Updating incorrect values in datasets.
- Modifying selected elements based on conditions.
- Data preprocessing in search-and-replace tasks.

- **Conclusion:**

This simple yet effective algorithm demonstrates how to perform a search-and-replace operation on an array. By scanning each element and replacing matches, we efficiently update data without extra memory usage. This operation is fundamental in many real-life programming tasks and data processing pipelines.

Question 3 :

Answer: A queue is a linear data structure that follows the principle of **FIFO** – *First In, First Out*. This means that the element which is inserted first will be the first one to be removed. It works just like a real-world queue where people stand in line: the one who comes first gets served first.

Basic Operations in a Queue:

1. **Enqueue:**
Adding (inserting) an element at the rear (end) of the queue.
2. **Dequeue:**
Removing an element from the front of the queue.
3. **Peek/Front:**
Viewing the element at the front without removing it.

4. **IsEmpty / IsFull:**

Used to check whether the queue is empty or full (important in array-based queues).

Representation of a Queue:

A queue can be implemented using:

- **Arrays**
- **Linked Lists**

Example using an array:

- Front $\rightarrow$ [10, 11, 12, 13] $\leftarrow$ Rear
- Enqueue 14 $\rightarrow$ [10, 11, 12, 13, 14]
- Dequeue $\rightarrow$ removes 10 $\rightarrow$ [11, 12, 13, 14]

Real-Life Examples of Queue:

1. Bank Queue System:

Customers take a token and are served in the order they arrive. The first customer in line is the first to be served.

2. Print Spooling:

When multiple documents are sent to a printer, they are printed in the order they were received.

3. Ticket Counters:

Whether at a railway station or a movie theatre, people stand in a queue and are served in order.

4. Call Center Support:

Customer support calls are answered in the order they are received — older calls are answered first.

5. CPU Scheduling (Round Robin):

In operating systems, tasks are often scheduled in a queue where the CPU serves processes in order.

Advantages of Queue:

- Simple and easy to implement.
- Maintains order of processing.
- Used widely in computer science and real-world systems.

Applications in Data Structures:

- Used in Breadth-First Search (BFS) in graphs.
- Implementing buffers like keyboard or network buffers.
- Managing requests in web servers and operating systems.

Conclusion:

A queue is a fundamental structure used to manage data in a fair and orderly manner. By using FIFO, it ensures that every element gets a chance to be processed in the correct order. Whether in real-life situations or system-level operations, queues are essential for managing data efficiently and predictably.

Set-2

Question 4 :

Answer : A linked list is a linear data structure in which elements, called nodes, are stored in non-contiguous memory locations. Each node contains two parts:

1. Data – the value to be stored.
2. Pointer – the address of the next node.

Unlike arrays, linked lists do not require a fixed size and allow dynamic memory allocation.

- **Structure of a Node:**

```
struct Node {
    int data;
    Node* next;
};
```

Types of Linked Lists:

1. Singly Linked List:

- Each node points to the next node in the sequence.
- The last node points to NULL.
- Traversal is only forward.

Example:

[23 | *] → [24 | *] → [25 | NULL]

2. Doubly Linked List:

- Each node contains two pointers: one to the next node and one to the previous.
- Allows traversal in both directions.

Example:

NULL $\leftarrow$ [23] $\leftrightarrow$ [24] $\leftrightarrow$ [25] $\rightarrow$ NULL

3. Circular Linked List:

- The last node points back to the first node.
- Can be singly or doubly circular.

Example: [23] $\rightarrow$ [24] $\rightarrow$ [25] $\rightarrow$ back to [23]

- **Advantages of Linked List over Array:**

Feature	Array	Linked List
Size	Fixed at declaration time	Dynamic (can grow or shrink)
Memory Allocation	Continuous	Non-continuous
Insertion/Deletion	Costly (requires shifting)	Efficient (only pointer adjustment)
Wastage of Space	May have unused space	No wastage, uses memory as needed
Implementation	Simple but rigid	Slightly complex but flexible

Key Benefits:

- Efficient memory usage.
- Easy insertion and deletion at any position.
- No need to shift elements like in arrays.
- Ideal for implementing stacks, queues, graphs, and other dynamic data structures.

When to Use Linked List:

- When you don't know the size of the data in advance.
- When frequent insertions/deletions are required.
- When working with dynamic memory structures like queues or trees.

Conclusion:

A linked list is a flexible and dynamic data structure that solves many limitations of arrays. It allows efficient memory usage and provides better performance for insertion and deletion operations. By understanding its types — singly, doubly, and circular — programmers can apply the right kind of linked list for different problems.

Question 5 :

Answer : A Doubly Circular Queue is a special type of queue that combines the features of both a doubly linked list and a circular queue. It is a linear data structure where:

- Each node has two pointers: one pointing to the next node and one to the previous.
- The last node is connected to the first node, and the first node is connected to the last, forming a closed loop in both directions.

Structure of a Node:

```
struct Node {  
    int data;  
    Node* next;  
    Node* prev;  
};
```

Features of a Doubly Circular Queue:

1. **Bi-directional Traversal:**
You can move both forward and backward through the queue.
2. **Circular Nature:**
The last node's next pointer points to the first node, and the first node's prev pointer points to the last node.
3. **Efficient Insertion/Deletion:**
Possible at both front and rear ends without shifting elements.

Applications:

- Used in task scheduling where we need to loop through tasks in both directions.
- Useful in multiplayer gaming systems, where turns are rotated.
- Works well in memory management systems.

Algorithm to Display Elements of a Doubly Circular Queue

Let front be the pointer to the front of the queue.

Algorithm DisplayDoubleCircularQueue(front)

Step 1: Start

Step 2: If front is NULL

 Print "Queue is empty"

 Stop

Step 3: Set $\text{temp} \leftarrow \text{front}$

Step 4: Repeat

Print $\text{temp} \rightarrow \text{data}$

$\text{temp} \leftarrow \text{temp} \rightarrow \text{next}$

Until $\text{temp} == \text{front}$

Step 5: Stop

Example:

Assume the queue has elements: $17 \leftrightarrow 18 \leftrightarrow 19$

- Front points to 17
- Rear (last node) points to 19
- $19 \rightarrow \text{next} = 17, 17 \rightarrow \text{prev} = 19$

Output:

Elements in Queue: 17 18 19

Time Complexity:

- **Display Operation:** $O(n)$
(We visit each node exactly once)

Conclusion:

A doubly circular queue is a powerful variation of queue that allows traversal and operations in both directions, with the added benefit of circular continuity. It is especially useful in systems where data must be accessed cyclically and efficiently from either end.

Question 6 :

Answer : Merge Sort is a popular and efficient comparison-based sorting algorithm that follows the Divide and Conquer strategy. It works by dividing the input array into smaller sub-arrays, sorting them, and then merging the sorted sub-arrays to form the final sorted array.

Divide and Conquer Approach in Merge Sort:

The Divide and **Conquer** technique involves three main steps:

1. **Divide:**
Split the array into two halves (left and right).
2. **Conquer:**
Recursively sort both halves.
3. **Combine (Merge):**
Merge the two sorted halves into a single sorted array.

This process is repeated until the whole array is sorted.

Merge Sort Algorithm

Algorithm MergeSort(arr, left, right)

Step 1: If left < right then

- a. Find middle = (left + right) / 2
- b. Call MergeSort(arr, left, middle)
- c. Call MergeSort(arr, middle+1, right)
- d. Call Merge(arr, left, middle, right)

Step 2: End If

Step 3: Stop

Merge Function (Combining Step)

Algorithm Merge(arr, left, mid, right)

Step 1: Create temporary arrays L and R

Step 2: Copy elements into L and R from arr

Step 3: While L and R are not empty

Compare and copy the smaller element into original array

Step 4: Copy remaining elements (if any)

Step 5: Stop

Example:

Given Array:

[38, 27, 43, 3, 9, 82, 10]

Step-by-Step Process:

Divide: [38, 27, 43] and [3, 9, 82, 10]

Divide Again: [38], [27, 43], [3, 9], [82, 10]

Sort Each: [27, 38, 43] and [3, 9, 10, 82]

Merge: [3, 9, 10, 27, 38, 43, 82]

Time & Space Complexity:

Case Time Complexity

Best $O(n \log n)$

Case Time Complexity

Average $O(n \log n)$

Worst $O(n \log n)$

- **Space Complexity:** $O(n)$ (for temporary arrays)

Advantages of Merge Sort:

- Very predictable performance: always $O(n \log n)$
- Stable sort: maintains the order of equal elements
- Works well for large datasets

Conclusion:

Merge Sort is a powerful sorting algorithm based on the Divide and Conquer technique. It splits the problem into smaller sub-problems, solves them individually, and then combines them. This makes it highly efficient and reliable for handling large data sets.